

Evaluation of IPC through Linux Pipes

Patrick DeCabooter and Shaw Liu

1 Introduction

The UNIX pipe is the oldest IPC mechanism still in daily use, and its cost model is usually taken on faith: sixteen pages of buffer, writers block when it fills, larger writes amortize better. We measured those claims and then asked where the time goes.

The dominant variable turns out to be *placement*, not payload size. Two processes exchanging four bytes cost 708 ns per message on one core and 2042 ns across two, for identical work and identical system-call counts; the difference is entirely time spent descheduled. The same asymmetry governs bandwidth: the familiar throughput collapse at the pipe capacity happens only across cores. We establish capacity as its cause by varying it directly.

2 Apparatus and method

An Ubuntu 24.04 guest on a custom Linux 7.2.4 kernel (arm64), eight vCPUs, 12 GiB RAM, under QEMU/hvf on an Apple M1. Page size is 4 KiB and pipe-max-size is 1 MiB, so the resize sweep below runs unprivileged. Benchmarks are C at -O3 -Wall -Wextra. Absolute figures include hypervisor overhead; the comparisons between placements do not depend on it.

Processes are placed with sched_setaffinity after the fork, not with taskset, which imposes a single shared mask and cannot express “parent on core 0, child on core 1”. Each configuration is launched five times; each launch reports its internal minimum and we take the minimum again across launches, since noise can only add time. Throughput is a rate, so we report the maximum and never an arithmetic mean.

3 Choosing a clock

Variables. Cause: the clock API. Effects: accuracy, call cost, granularity. **Hypothesis.** clock_gettime can resolve a single exchange; gettimeofday cannot.

We compared clock_gettime(CLOCK_MONOTONIC), gettimeofday, and cntvct_el0, this architecture’s analogue of rdtsc. Timing sleep(5) five times, all three recovered 5.0024 s and agreed within 53 ppm; the 492 ppm excess is sleep’s own wakeup latency, which is precisely why the clocks agree on it.

Granularity came from 10^6 back-to-back reads into a preallocated array, post-processed afterwards, with nothing else in the loop (Table 1). The decisive column is the last: 97.45 % of adjacent gettimeofday readings are *identical*, two calls finishing inside one microsecond tick. A clock that cannot separate events by its own call cost cannot measure a 708 ns round trip, so we discarded it.

The cycle counter is cheapest to read but not finest: cntfrq_el0 reports 24 MHz, a 41.67 ns period. clock_gettime shows that same 41 ns quantum because current_clocksource is arch_sys_counter — the vDSO reads that very register and scales it. It costs three times more per call and gives up no resolution, so we used it throughout. Neither standard cycle-counter caveat bites here: frequency invariance follows from the sleep agreement, and reading cntvct_el0 on each of eight cores showed only 400 ns of noise with no systematic step, the arm64 generic timer being system-wide rather than per-core.

4 Latency

Variables. Cause: payload size and placement. Effect: one-way latency, taken as half a round trip so only one clock on one core is involved. **Hypothe-**

Clock	Call (ns)	Res. (ns)	Zero deltas
clock_gettime	35.3	41.0	19.83 %
gettimeofday	25.7	1000.0	97.45 %
cntvct_el0	11.3	41.7	73.69 %

Table 1: Clock granularity over 10^6 consecutive reads. “Zero deltas” is the fraction of adjacent readings the clock could not tell apart.

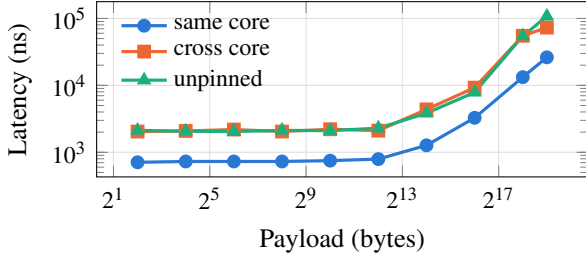


Figure 1: One-way latency (half round trip), minimum of five runs. The flat region below 4 KiB is fixed cost per exchange; the constant gap between same-core and cross-core is 1333 ns.

sis. Flat while fixed cost dominates, then linear once copying does.

Figure 1 confirms that shape and adds one the hypothesis missed. Below 4 KiB every curve is flat: cost is per exchange, not per byte. Above 16 KiB growth is linear, and the same-core slope implies 19.2 GiB/s of copy bandwidth — a round trip copies the payload twice while the plotted value is half a round trip, so the factors of two cancel.

The surprise is the constant offset in the flat region: 708 ns same-core against 2042 ns cross-core, a payload-independent penalty of 1333 ns. Unpinned runs track the cross-core curve, so the scheduler spreads the pair by default. Section 6 accounts for that penalty.

5 Throughput

Variables. Cause: chunk size and placement. Effect: sustained one-way bandwidth, acknowledged once for the whole transfer. **Hypothesis.** Throughput rises as the fixed per-write cost amortizes, then falls once chunks exceed capacity and the writer blocks.

Figure 2(a) shows the first half right and the second wrong. Only the cross-core curve collapses, peaking

at 6431 MiB/s at 8 KiB and falling to 2391 MiB/s by 512 KiB. The same-core curve climbs to 9678 MiB/s and stays flat through chunks eight times the capacity. That falsifies blocking as the explanation: the same-core writer blocks just as often, the pipe being the same size, and pays nothing for it.

The mechanism is loss of *overlap*. Across cores, writer and reader truly run in parallel, and that pipelining survives only while the writer can deposit a whole chunk without stalling. Past capacity it stalls mid-chunk every time, the pair falls into lockstep, and every handoff costs a full cross-core wakeup. On one core there was no parallelism to lose, so bandwidth simply saturates at what the cache hierarchy sustains. The crossover at 4 KiB follows: below it overlap beats locality, above it locality wins.

6 Where the time goes

Filling a nonblocking pipe until EAGAIN accepts exactly 65,536 bytes, matching both `F_GETPIPE_SZ` and the sixteen-page folklore; single-byte writes reach the same total, so no page is wasted per write.

Table 2 accounts for the wall clock with `getrusage`. At 4 B the same-core pair spends essentially all its time in the kernel and none blocked. The cross-core pair does identical work — `strace` confirms exactly two write and two read calls per round trip — yet is ten times slower, with 61 % of its wall clock in neither user nor system time. That residue answers the 1333 ns question: it is time descheduled, awaiting a wakeup on another core.

The context-switch column closes the loop with capacity: 1.99 voluntary switches per round trip at 4 B and 15.99 at 512 KiB, exactly $2 \lceil \text{size}/\text{capacity} \rceil$ against the measured 64 KiB. `strace` gives the same arithmetic from the other side — at 512 KiB the reader issues nine read calls per round trip, eight for data and one for the ack, because a read returns only what is buffered. Capacity sets the number of handoffs; placement sets the price of each.

Finally we made capacity the independent variable. Figure 2(b) re-runs the cross-core sweep with `F_SETPIPE_SZ` from 4 KiB to 1 MiB. The knee follows it: a 16 KiB pipe collapses at 8 KiB chunks, a 64 KiB pipe at 64 KiB, a 256 KiB pipe at 256 KiB, and a 1 MiB pipe, larger than any chunk tested, never

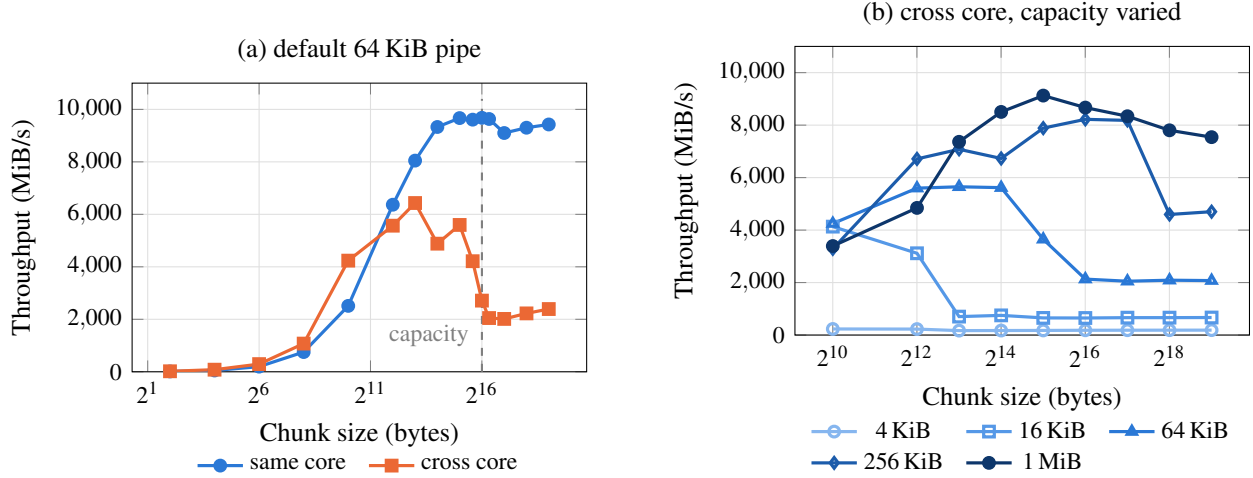


Figure 2: (a) Streaming throughput at the default capacity. Only the cross-core curve collapses; the same-core curve saturates and stays flat well past capacity. (b) With `F_SETPPIPE_SZ` varying the capacity, the cross-core knee tracks the configured size, establishing it as the cause.

Payload	Place	$\mu\text{s}/\text{RT}$	%usr	%sys	%blk	cs/RT
4 B	same	2.3	0	101	0	1.99
4 B	cross	23.0	3	37	61	2.00
512 KiB	same	58.9	14	88	0	15.99
512 KiB	cross	346.8	0	53	47	15.99

Table 2: Where the wall clock goes, per round trip. Percentages are `getrusage` totals for both processes against elapsed time and may exceed 100 % by a point from tick granularity. “cs/RT” is voluntary context switches per round trip.

collapses. The correlation is causal.

7 Conclusions

Pipe cost is set by two quantities, and payload size is only one of them. Capacity fixes how many hand-offs a transfer needs, exactly $2^{\lceil \text{size}/\text{capacity} \rceil}$ volun-

tary context switches, confirmed against a measured 64 KiB and then verified causally by moving it. Placement fixes the price of each handoff: 1333 ns of de-scheduled time when the peer is on another core, three quarters of the total at small payloads.

The practical consequence is counterintuitive. Spreading a communicating pair across cores buys parallelism worth having only while messages stay below capacity; past that, the pair is four times faster confined to a single core. A benchmark that leaves placement to the scheduler measures the cross-core case without saying so.

Two limits bound these results: everything ran virtualized, so absolute latencies carry hypervisor overhead, and no PMU was available in the guest, so Table 2 rests on `getrusage` and `strace` rather than hardware counters. We report the residue as blocked because it is neither user nor system time, not because a counter placed it there.